



Selección y validación de modelos

Aprendizaje Automático 1 • Clase 11



El recorrido de hoy

1. **¿Contra qué comparo?** Modelos base.
2. **¿Cómo elijo los hiperparámetros?** Grid, random y búsqueda bayesiana.
3. **¿Cómo elijo el modelo final?** Comparación más allá de la métrica.
4. **¿Y si automatizo todo esto?** AutoML.



Modelos base

Baseline model



Qué es un modelo base

Son modelos simples y por lo general **ingenuos**: no aprenden casi nada.

Se usan como **punto de partida**. Cualquier modelo que desarrolle el equipo tiene que ganarle, y por un margen que valga la pena.

Sirven para dos cosas: saber si el problema es fácil o difícil, y detectar errores (si el modelo base gana, algo está mal).

Son baratos de construir y se hacen **antes** de invertir tiempo en algo complejo.



Modelos base para clasificación

Basado en la mayoría: clasificar todo en la clase mayoritaria. Es el más útil en problemas desbalanceados.

Clasificador aleatorio: asignar las clases al azar, respetando o no las proporciones.

Basado en un criterio ingenuo: por ejemplo, para clasificar una crítica como positiva o negativa, contar palabras positivas contra negativas.

Benchmarks: si el problema es conocido, el resultado publicado por otros también funciona como piso.



Para regresión y series de tiempo

Regresión: predecir siempre la media. O la mediana, que aguanta mejor los valores extremos. O cualquier otro estadístico.

Regresión lineal simple: usar una sola característica, la más obvia, para predecir.

Series de tiempo: predecir el último valor observado. Se lo llama **naive forecast** y es sorprendentemente difícil de ganar.

Series con estacionalidad: predecir el valor del mismo período del año anterior.



Optimización de hiperparámetros



Hiperparámetros

Son aquellos parámetros seteados manualmente antes de comenzar a entrenar un modelo.

En el caso de gradiente descendiente, puede ser el ratio de aprendizaje.

En el caso de Ridge o Lasso, puede ser el parámetro de regularización.

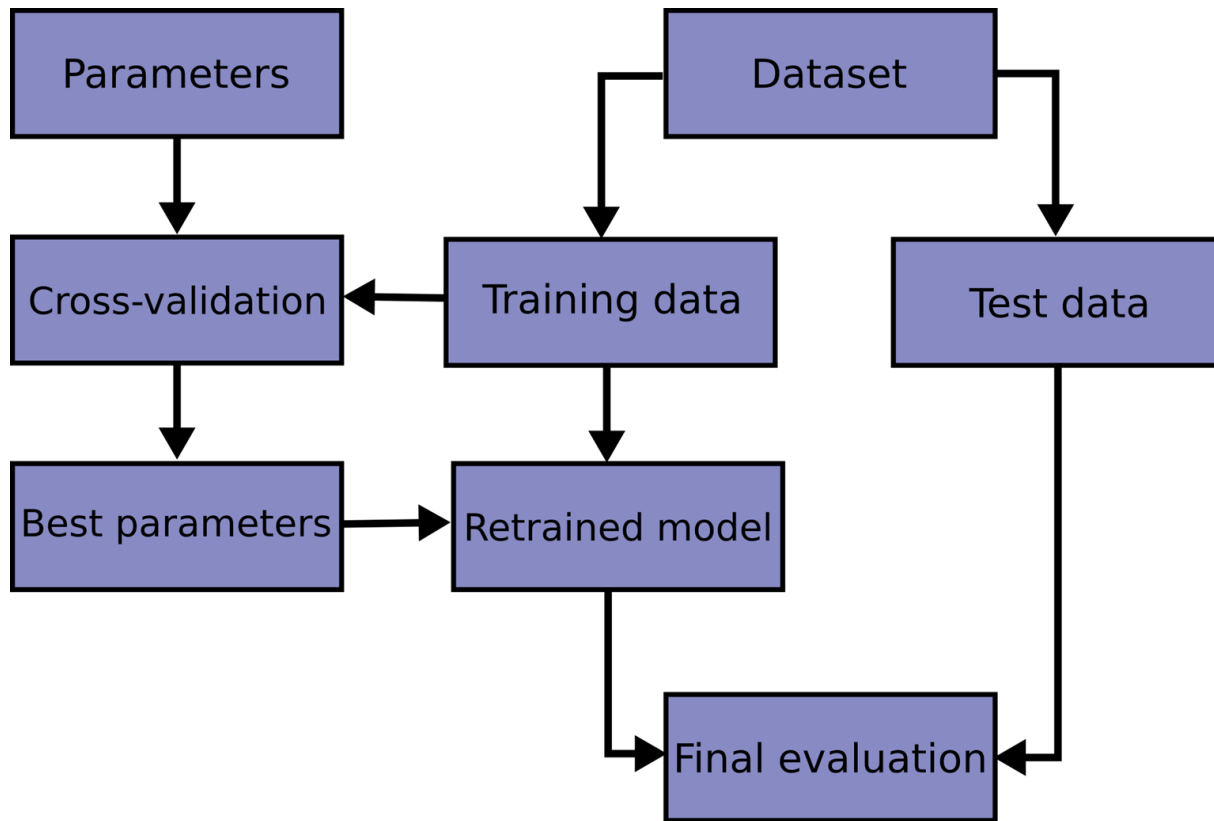
¿Conocen más?



Conjunto de validación

A la hora de evaluar hiperparámetros es bueno contar con un conjunto de validación, además de entrenamiento y prueba.

Se entrena el modelo con el conjunto de entrenamiento y determinados valores de hiperparámetros. Se utiliza el conjunto de validación para ver las métricas obtenidas. Se prueba con otro conjunto de hiperparámetros y así sucesivamente. Nos podemos quedar con los hiperparámetros que den mejores métricas en el conjunto de validación. Luego se hace el test en el conjunto de prueba.





Qué necesita una búsqueda

Un estimador: el modelo, o mejor el pipeline completo.

Un espacio de valores: qué hiperparámetros variar y en qué rango.

Un método de búsqueda: cómo muestrear ese espacio.

Un esquema de validación cruzada: cómo se mide cada combinación.

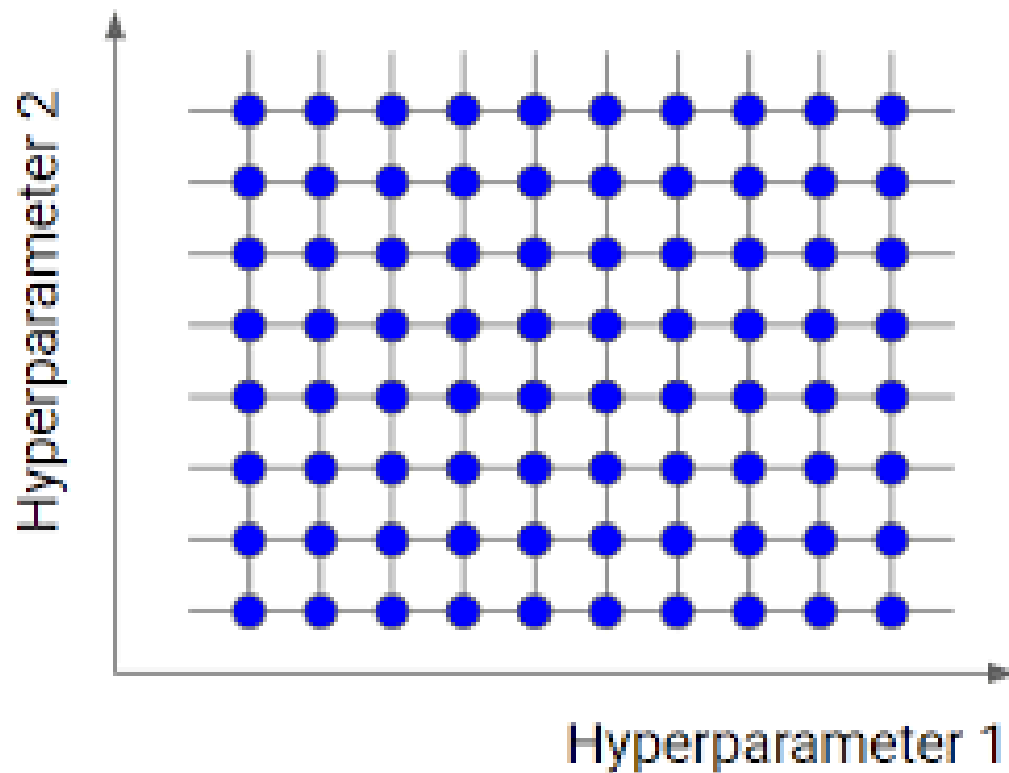
Una métrica a mejorar: la que decidimos que representa el problema.



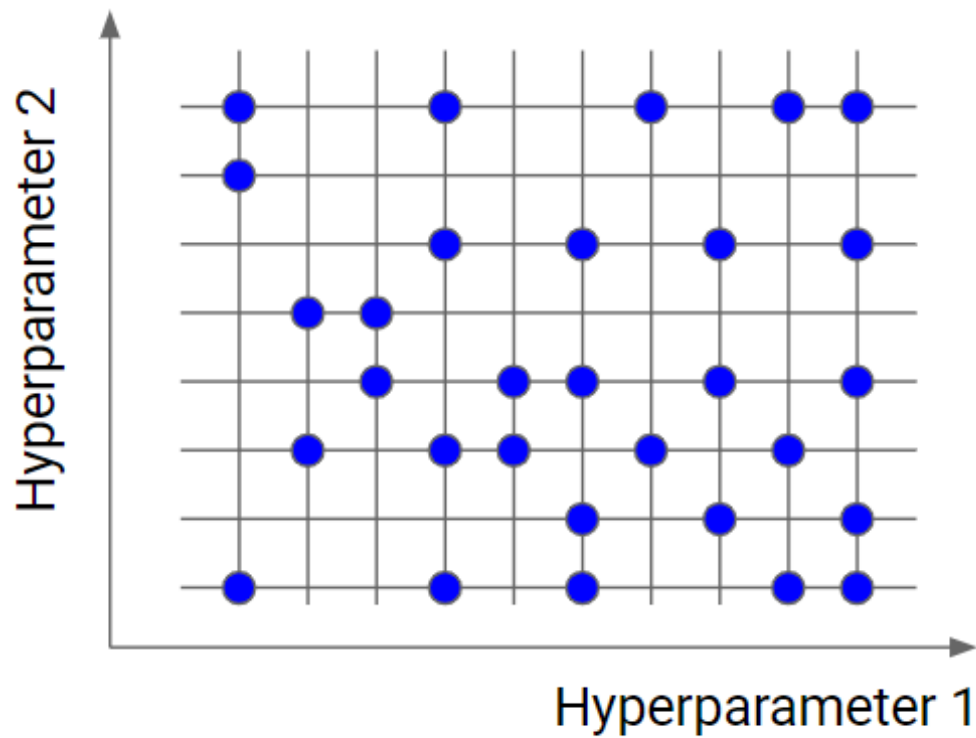
Métodos de búsqueda

- Grid Search
- Random Search
- Sequential Search con enfoque Bayesiano

Grid search



Random search





Cuándo usar cada uno

Grid search: prueba todas las combinaciones de una grilla. Exhaustivo y reproducible, pero explota: 4 hiperparámetros con 5 valores son 625 combinaciones, y por 5 folds, 3125 entrenamientos.

Random search: muestrea al azar una cantidad fija de combinaciones. Vos elegís el presupuesto.

Con **uno o dos** hiperparámetros, la grilla completa está bien. Con **tres o más**, conviene al azar.

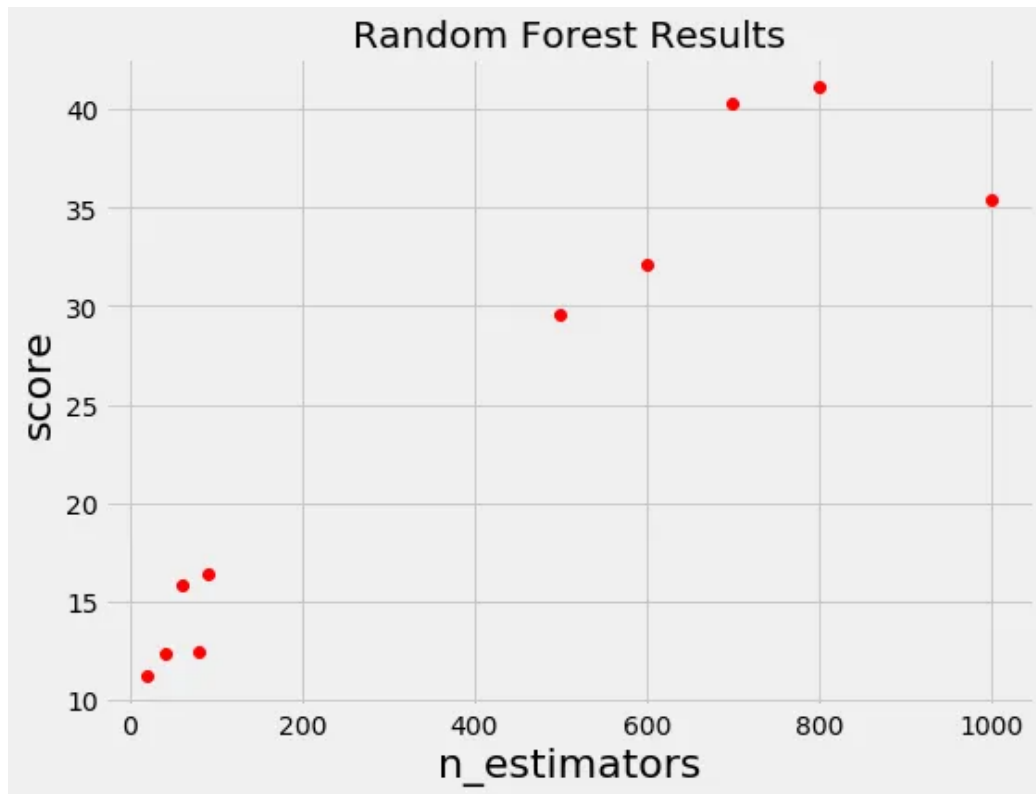
Además la búsqueda al azar admite distribuciones continuas, así que no te limita a los valores que se te ocurrieron.

Enfoque bayesiano

Supongan que a menor score, mejor el modelo, y que ya se corrieron distintos modelos variando el hiperparámetro “n_estimators”

¿Con qué valores de n_estimators seguirían probando?

¿Pondrían 800? ¿Y 100?





Cómo funciona la búsqueda bayesiana

Grid y random son **ciegas**: cada prueba es independiente y no aprenden nada de las anteriores.

La búsqueda bayesiana arma un modelo de cómo se relaciona cada hiperparámetro con la métrica, y lo usa para decidir dónde probar después.

Balancea **explorar** zonas desconocidas con **explotar** las zonas que ya vienen dando bien.

Rinde mucho más cuando cada entrenamiento es caro. La librería más usada hoy es **Optuna**.



Métodos de búsqueda

[Grid Search](#)

[Random Search](#)

[Colab introducción a Optuna](#)



Comparación de modelos



Sobre qué datos se compara

Para comparar modelos, todos tienen que entrenarse y evaluarse sobre **los mismos conjuntos**: el mismo train y el mismo test.

Y con los **mismos folds** de validación cruzada. En scikit-learn, con el mismo objeto de CV y la misma semilla.

Si cada modelo se evaluó con una partición distinta, las diferencias pueden ser sólo azar de la partición.

Acordate del desvío entre folds: si dos modelos difieren menos que ese desvío, la diferencia probablemente no sea real.



El problema de elegir la métrica

Hay que elegir **una** métrica que represente el problema, y ordenar los modelos por ella.

Elegir varias trae un problema: si el modelo X tiene 0,8 en la métrica A y 0,9 en la B, y el modelo Y las tiene al revés, ¿cuál es mejor?

La salida práctica: definir una métrica principal **antes** de ver los resultados, y usar las otras como restricciones (“maximizar recall con precisión por encima de 0,6”).

Elegirla después de ver los números es elegir el modelo que querías y justificarlo.



Las otras cuatro dimensiones

Complejidad y varianza. Los algoritmos más complejos suelen dar mejor métrica pero con más varianza. ¿Vale un modelo 1 % peor si tiene 10 % menos de varianza?

Interpretabilidad. Cuanto más caja negra, menos se explica. A veces conviene ceder métrica para poder justificar las decisiones. Es la clase 10.

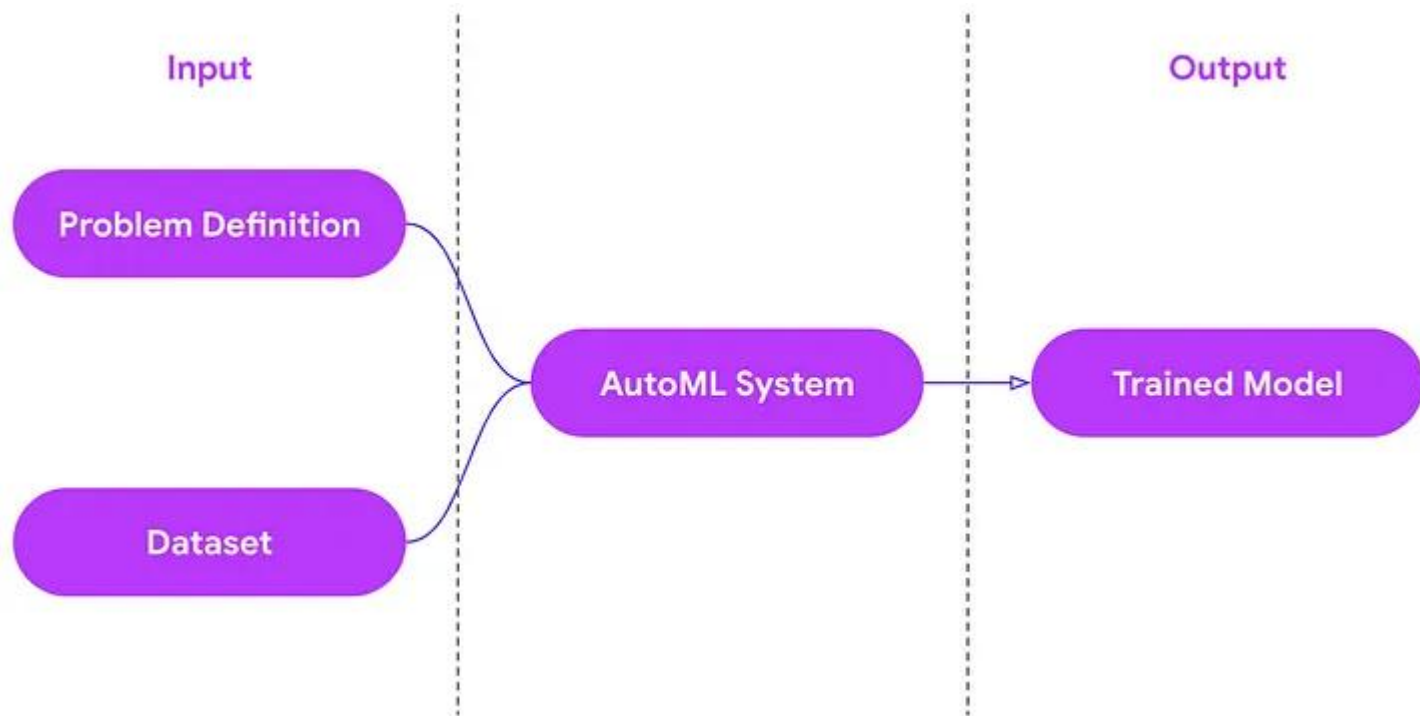
Costo computacional. No sólo entrenar la primera vez: hay que **reentrenar** periódicamente.

Latencia. ¿Y si la respuesta se necesita en milisegundos? Ahí hay modelos que quedan descartados por más que ganen.



AutoML

Aprendizaje de máquina automático





Qué es

AutoML toma la definición del problema y los datos de entrenamiento, y devuelve un modelo entrenado listo para implementar.

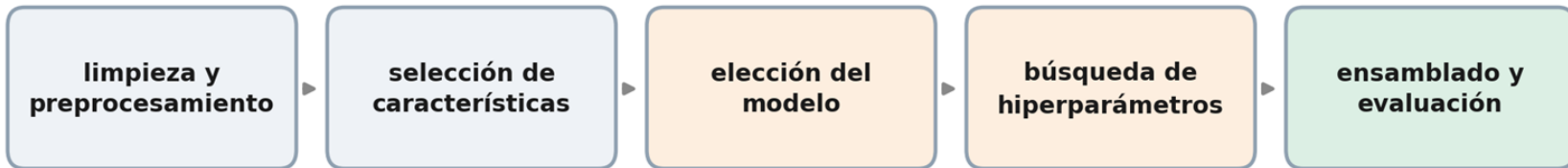
Por ejemplo: le das un conjunto de imágenes etiquetadas y te devuelve un clasificador de imágenes entrenado.

Por dentro hace exactamente lo que vimos en esta clase, pero automáticamente y probando muchísimas combinaciones.



Qué automatiza y qué no

El pipeline completo, buscado automáticamente



esto es lo que AutoML automatiza

Lo que NO automatiza: definir el problema, conseguir y entender los datos, elegir la métrica y decidir si el modelo se puede poner en producción.



AutoML

En resumen, el aprendizaje automático automatizado (AutoML) trata de cerrar la brecha entre los diversos modelos de aprendizaje automático sofisticados y las técnicas de entrenamiento disponibles hoy en día y los problemas del mundo real que podrían resolver, proporcionando soluciones end-to-end de manera automatizada.



Los límites

No te salva de la fuga de datos. Si hay una columna con información del futuro, AutoML la va a encontrar y explotar mejor que vos.

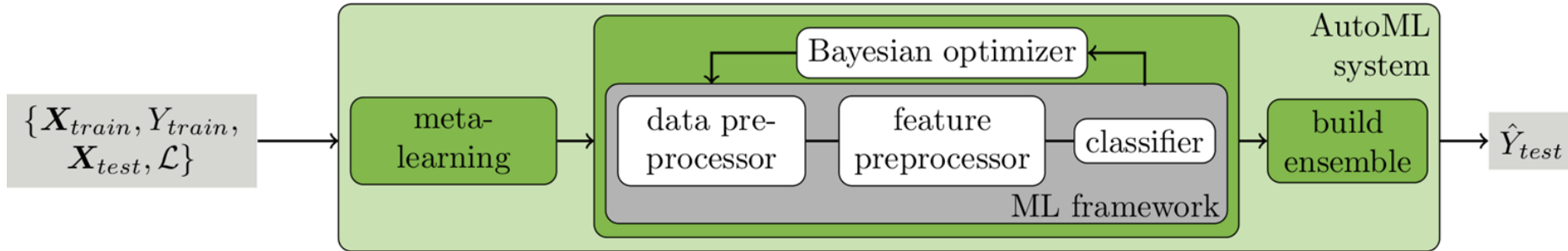
Cuesta caro. Una búsqueda seria puede ser horas de cómputo.

Suele dar modelos poco interpretables: ensambles de muchos modelos distintos.

No reemplaza el criterio. Sigue haciendo falta saber si el resultado tiene sentido, y sobre todo saber qué medir.

Sirve muy bien como **punto de referencia fuerte:** si tu modelo no le gana a AutoML, vale la pena preguntarse por qué.

Auto-sklearn



```
import autosklearn.classification
cls = autosklearn.classification.AutoSklearnClassifier()
cls.fit(X_train, y_train)
predictions = cls.predict(X_test)
```

GitHub TabPFN

Notebook TabPFN Demo

GitHub Auto-sklearn

Ejemplos Auto-sklearn